



中山大学 软件工程学院

SUN YAT-SEN UNIVERSITY SCHOOL OF SOFTWARE ENGINEERING

SSE206 《计算机网络》

第三章 传输层

南雨宏

nanyh@mail.sysu.edu.cn

2026-04-13

第 3 章：提纲

3.1 概述和传输层服务

3.2 多路复用与解复用

3.3 无连接传输：UDP

3.4 可靠数据传输的原理

3.5 面向连接的传输：TCP

- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

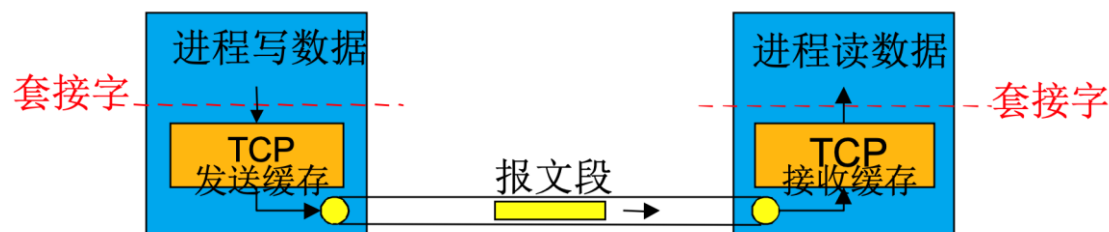
3.6 拥塞控制原理

3.7 TCP拥塞控制

3.8 传输层功能的演化

TCP: 概述 RFCs: 793, 1122, 2018, 5681, 7323

- 点对点 point-to-point:
 - 一个发送方，一个接收方
- 可靠的、按序的字节流:
 - 没有报文边界
- 全双工 full duplex data:
 - 在同一连接中数据流双向流动
 - 数据量受限于MSS: 最大报文段大小
 - 1460 bytes (应用层数据)
 - 以太网最大 MTU 1500 bytes (网络层数据)
- 累积确认 cumulative ACKs
- 管道化（流水线）：
 - 通过拥塞控制和流量控制设置窗口大小
- 面向连接:
 - 在数据交换之前，通过握手(交换控制报文)初始化发送方、接收方的状态变量
- 有流量控制:
 - 发送方不会淹没接收方



TCP 报文段结构

ACK: 下一个期待字节的序号
; 一个比特: 这是一个ACK

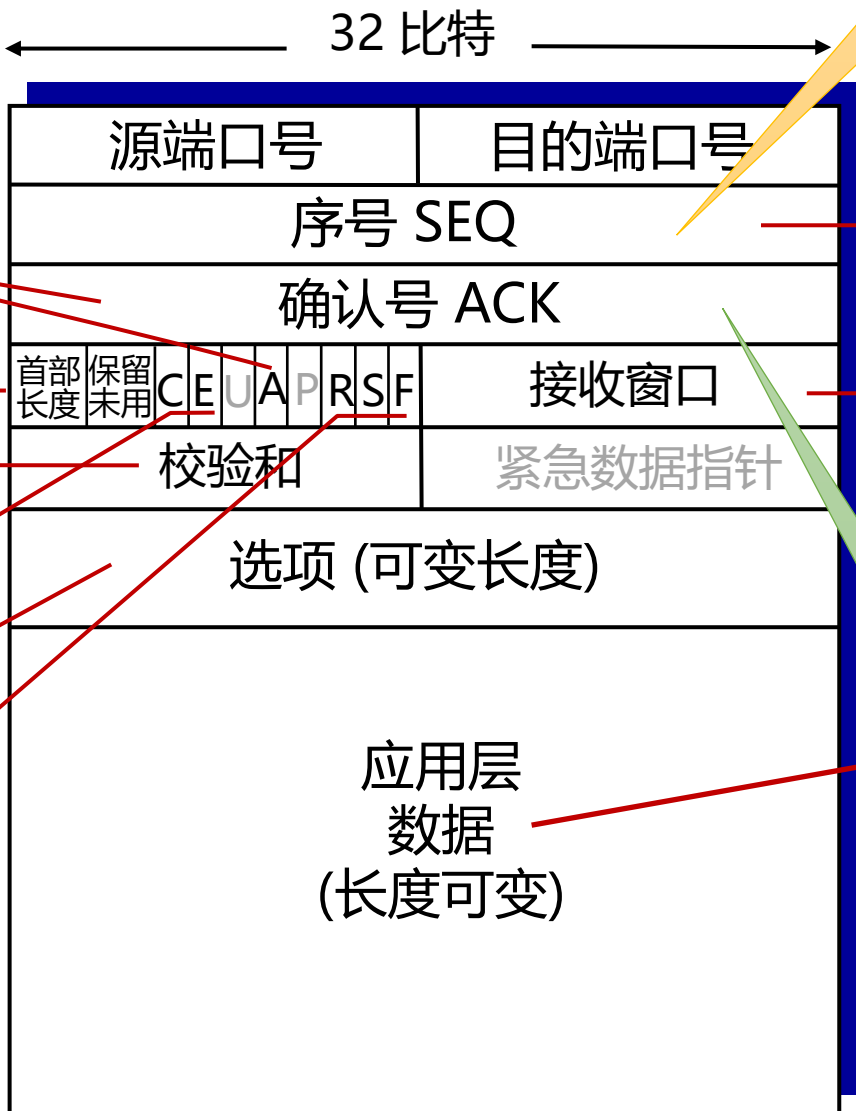
长度 (TCP首部)

Internet 校验和 (同UDP)

C, E: 拥塞通知

TCP 选项

RST, SYN, FIN: 连接管理 (建立/拆除连接)



字节序号

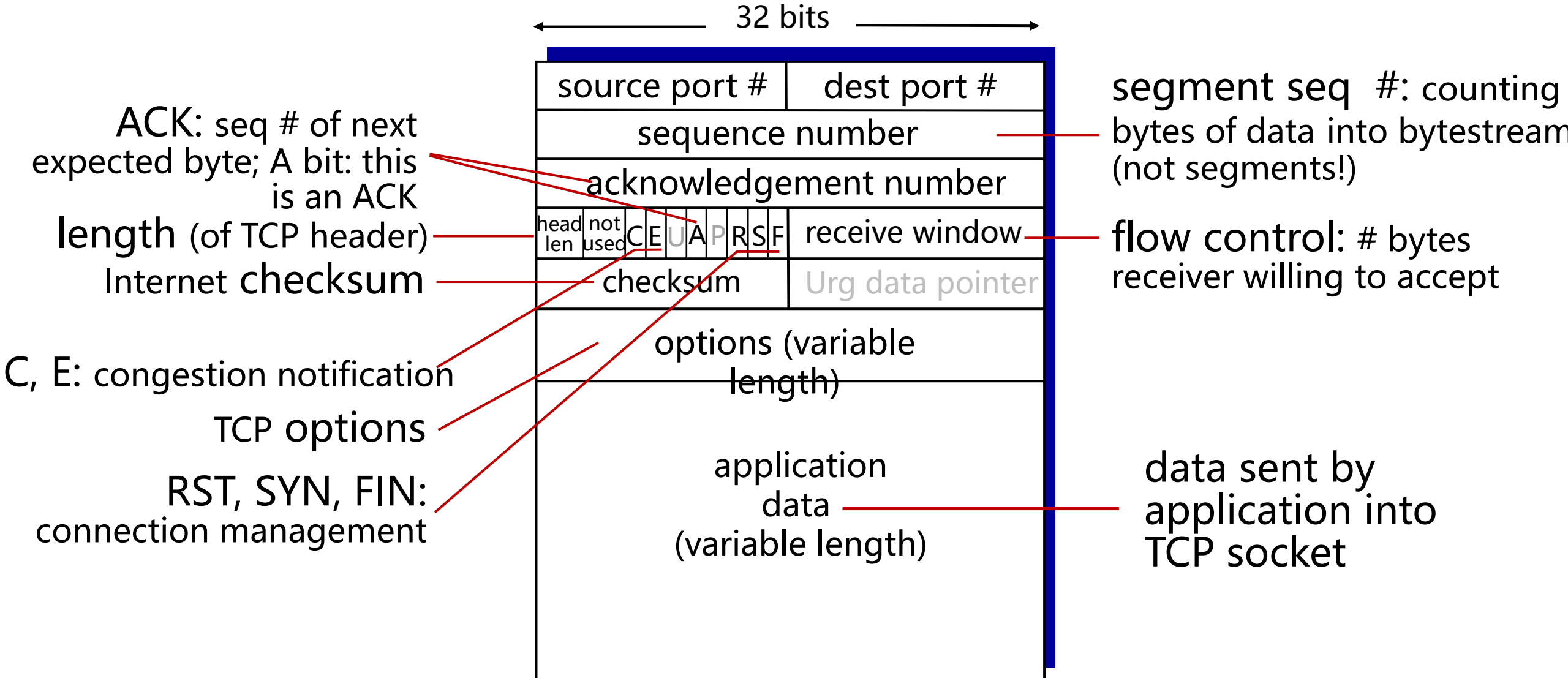
报文段序号: 将数据字节数计算到字节流中 (对字节计数, 而不是报文段!)

流量控制: 愿意接受的字节数量

应用程序发送到 TCP 套接字的数据

确认 n-1 及以前的字节

TCP segment structure



TCP 序号, 确认号

序号 Sequence numbers:

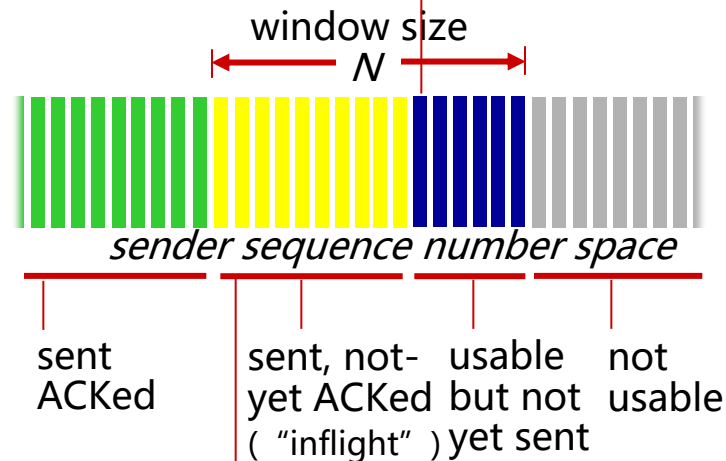
- 报文段首字节在字节流的编号

确认号 Acknowledgements:

- 期望从另一方收到的下一个字节的序号
- 累计ACK

发送方发送的报文段

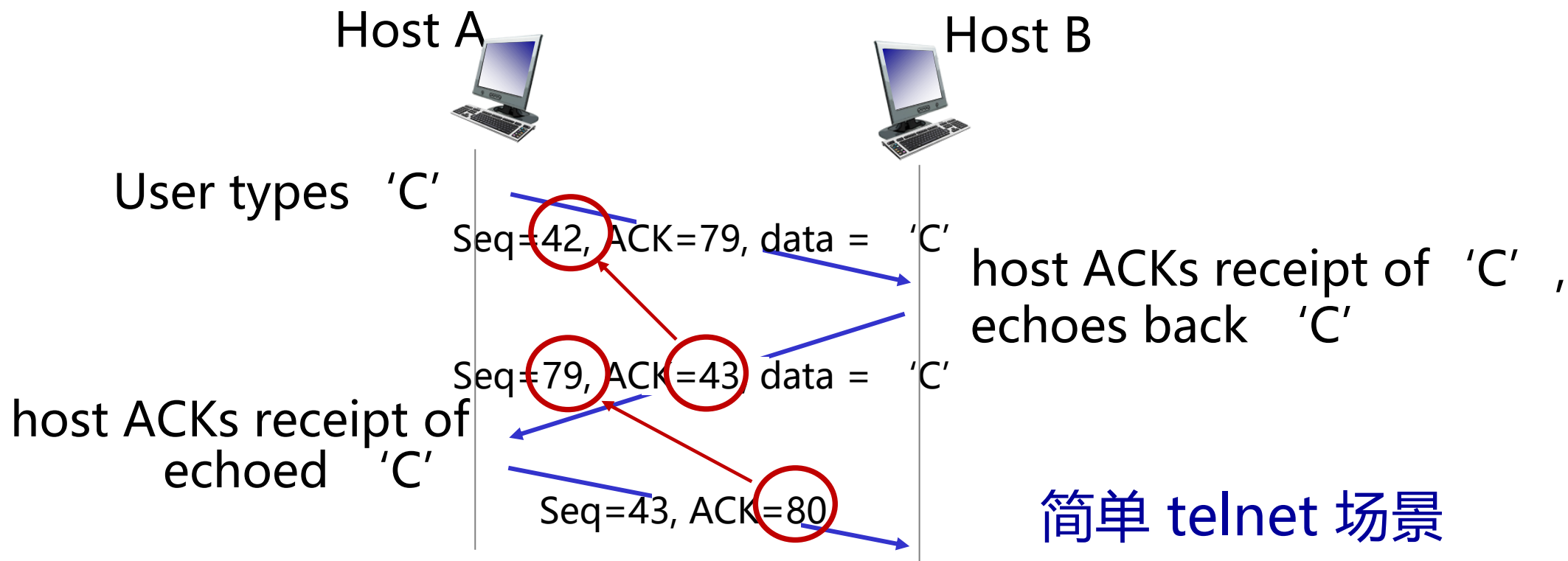
source port #	dest port #
sequence	
acknowledgement number	
	rwnd
checksum	urg pointer



发送方收到的报文段

source port #	dest port #
sequence number	
acknowledgement	
	A
checksum	urg pointer

TCP 序号和确认号



注意：TCP中，ACK表示**期望**的序号，而非已经接收到的最新序号

TCP 往返延时 (RTT) 和超时

■ Q 怎样设置 TCP 超时? (超时的时间怎么设置)

- 比RTT要长, 但 RTT 是变化的
- 太短: 太早超时, 不必要的重传
- 太长: 对报文段丢失, 反应太慢, 消极

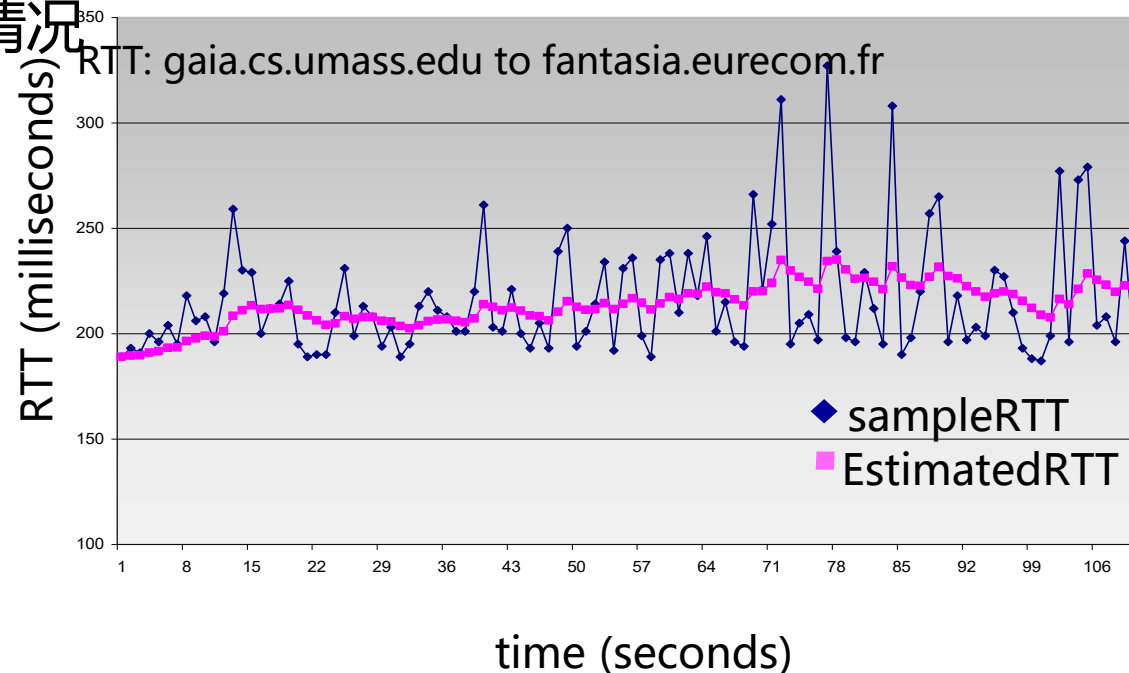
■ Q 怎么估计 RTT?

- SampleRTT (样本RTT) : 测量从报文段发出到收到确认的时间
 - 如果有重传, 忽略此次测量
- SampleRTT 会变化, 因此估计的 RTT 应该比较平滑
 - 对几个最近的测量值求平均 (EstimatedRTT), 而不是仅用当前的 SampleRTT

TCP 往返延时 (RTT) 和超时

$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- 指数加权移动平均 Exponential Weighted Moving Average (EWMA)
- 旧样本的影响呈指数衰减
- 越近的样本越能更好地反映当前网络情况
- 推荐值: $\alpha = 0.125$



TCP 往返延时 (RTT) 和超时

设置超时

- timeout interval = EstimatedRTT + “safety margin” 安全边界时间
 - EstimatedRTT 变化大 (方差大) → 需要较大的安全边界时间
 - SampleRTT 会偏离 EstimatedRTT 多远:

$$\text{DevRTT} = (1-\beta) * \text{DevRTT} + \beta * |\text{SampleRTT} - \text{EstimatedRTT}|$$

(推荐值, $\beta = 0.25$)

超时时间间隔设置为:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$$



↑
estimated RTT

↑
“safety margin”

第 3 章：提纲

3.1 概述和传输层服务

3.2 多路复用与解复用

3.3 无连接传输：UDP

3.4 可靠数据传输的原理

3.5 面向连接的传输：TCP

- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理

3.7 TCP拥塞控制

3.8 传输层功能的演化

TCP：可靠数据传输

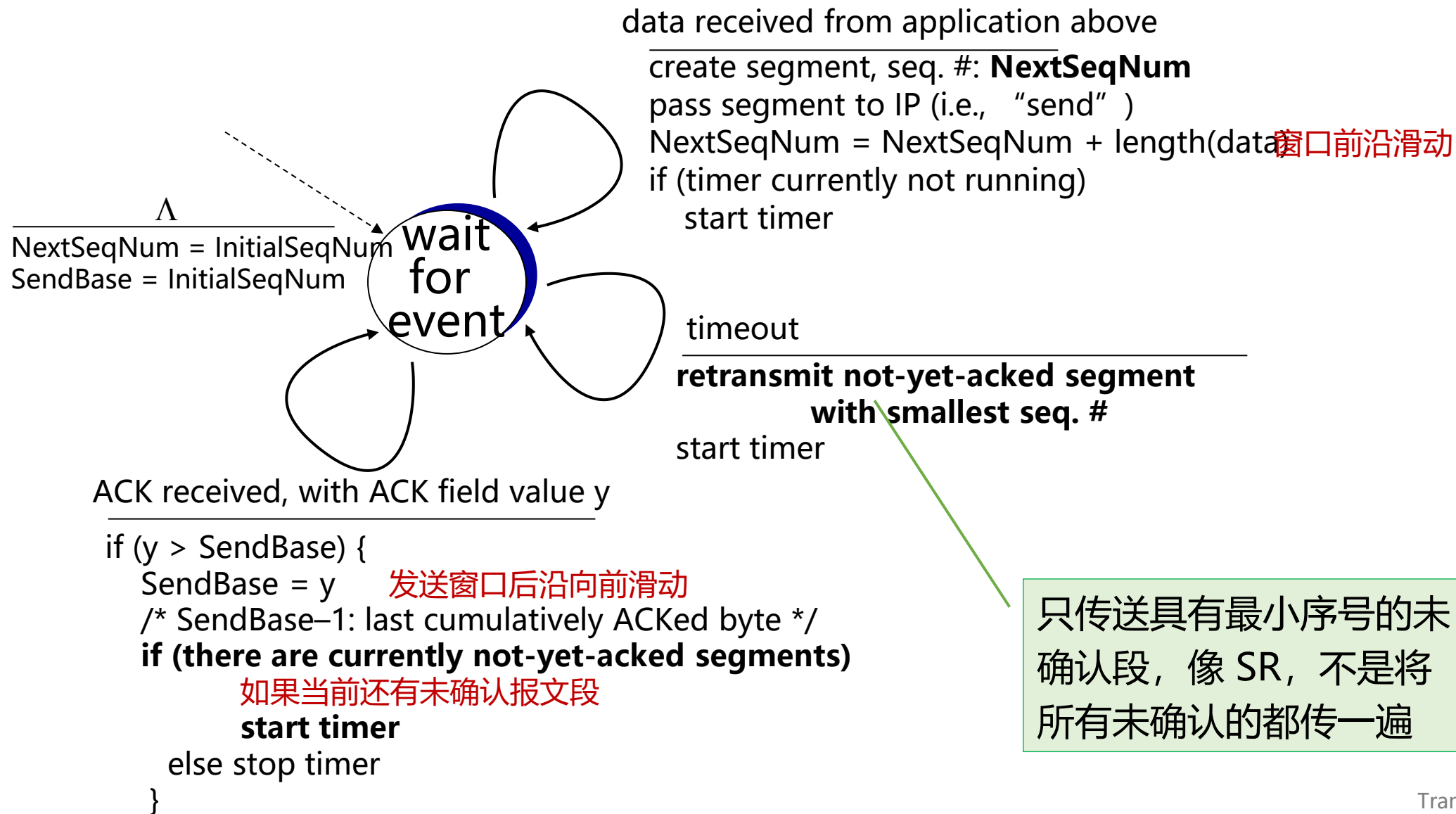
- TCP 在 IP 不可靠服务的基础上建立了 rdt
 - 管道化的报文段：批量发送 - GBN or SR
 - 累积确认（像GBN）
 - 单个重传定时器（像GBN）

Q: 接收方如何处理乱序的报文段

- A: TCP 没有规定，由实施者决定
- 可以缓存，也可以抛弃

- 通过以下事件触发重传
 - 超时（只重发最早的未确认段：SR）
 - 重复的确认（快速重传）
 - 例子：收到了ACK50，之后又收到3次ACK50
- 首先考虑简化的 TCP 发送方：
 - 忽略重复的确认
 - 忽略流量控制和拥塞控制

TCP 发送方 (简化版)



TCP 发送方（简化版）事件

事件：从应用层接收数据

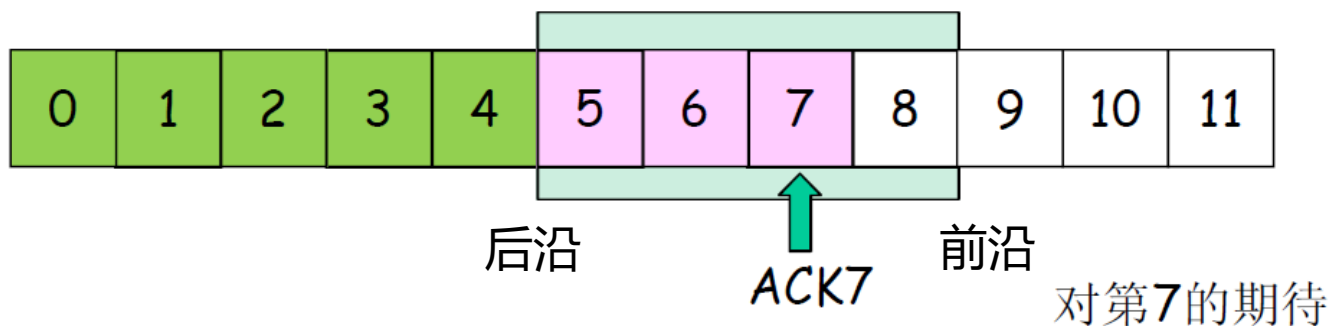
- 用 nextseq 创建报文段
- 序号 nextseq 为报文段首字节的字节流编号
- 如果还没有运行，启动定时器
 - 定时器与最早未确认的报文段关联
 - 过时间间隔: TimeoutInterval

事件：超时

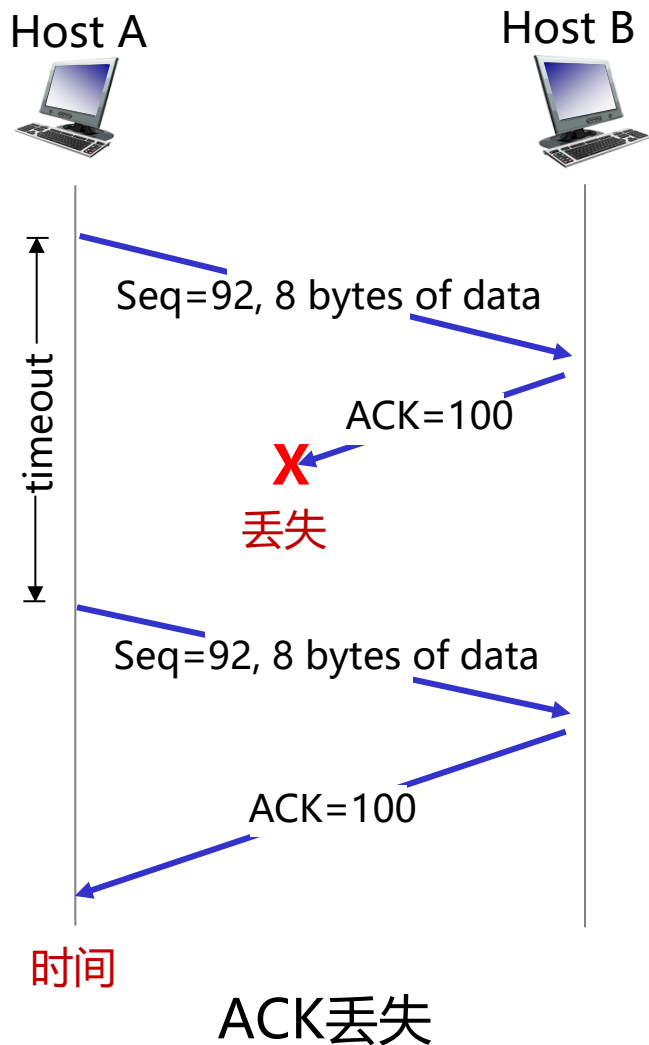
- 传后沿最老的报文段
- 重新启动定时器

事件：收到确认 ACK

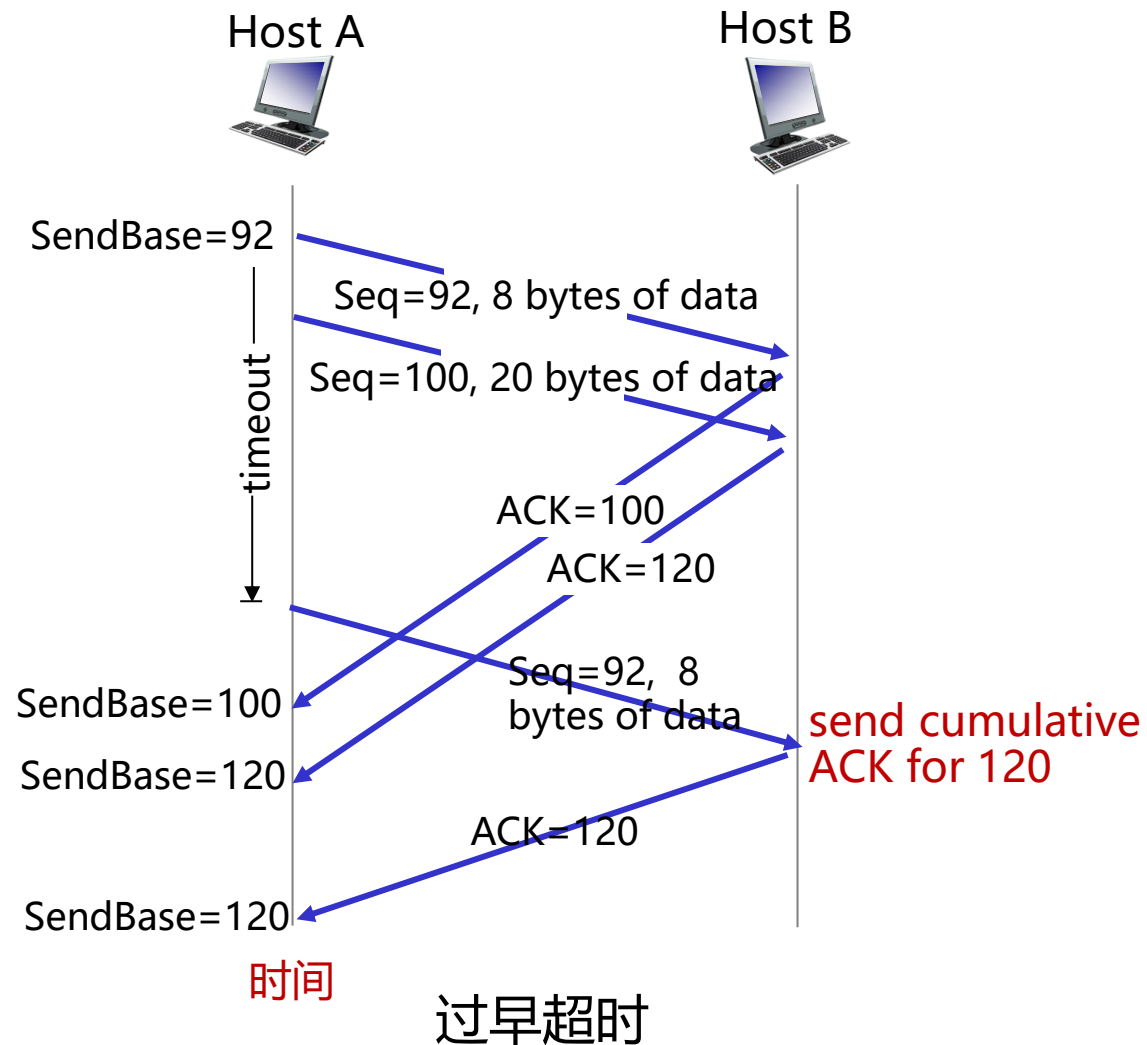
- 如果是对尚未确认的报文段确认
 - 更新已被确认的报文序号
 - 如果当前还有未被确认的报文段，重新启动定时器



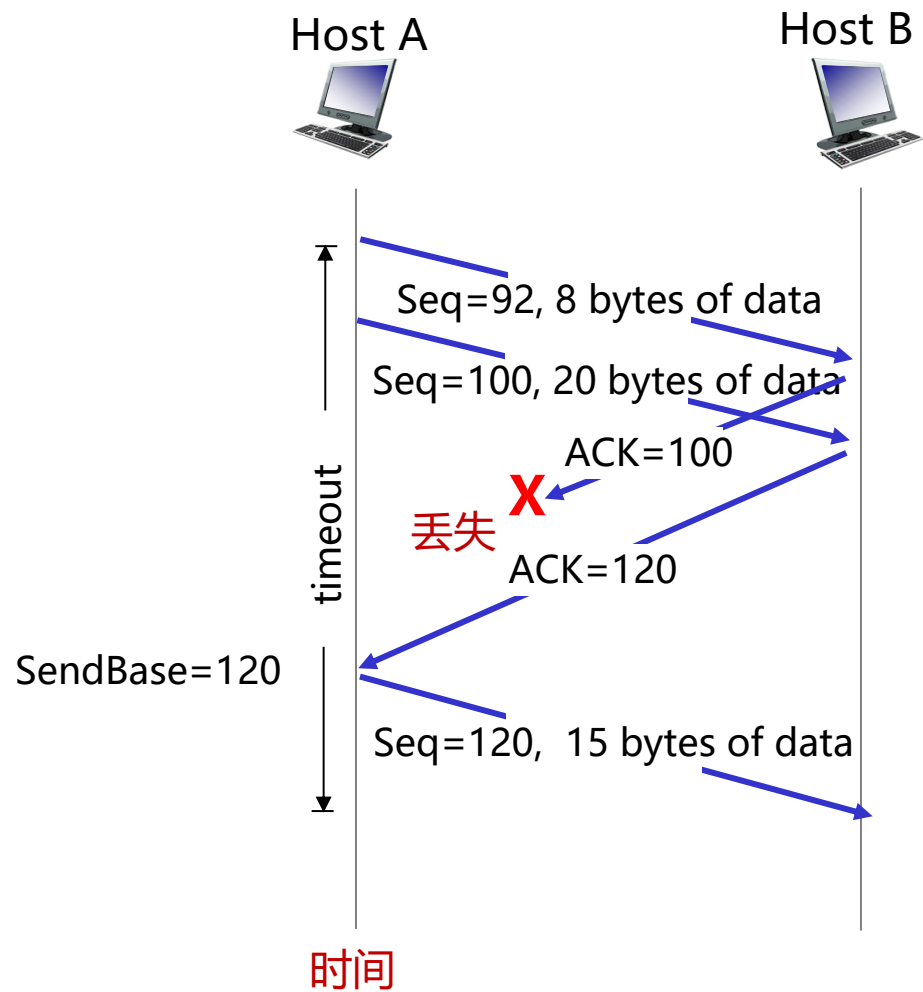
TCP: 重传



对顺序收到的最高字节确认



TCP: 重传



累积回复最新确认

TCP 接收方: 产生ACK的建议 [RFC 5681]

接收方事件

TCP 接收方动作

所期望序号的报文段按序到达。
所有在期望序号之前的数据都
已经被确认

延迟的ACK。对另一个按序报文段的到达
最多等待500ms。如果下一个报文段在这个
时间间隔内没有到达, 则发送一个ACK。

有期望序号的报文段到达。
另一个按序报文段等待发送ACK

立即发送单个累积ACK, 以确认**两个**按序
报文段。

比期望序号大的报文段乱序到达。
检测出数据流中的间隔

立即发送重复的ACK, 指明下一个期待字
节的序号

能部分或完全填充接收数据间隔
的报文段到达。

若该报文段起始于间隔 (gap) 的低端,
则立即发送ACK。

TCP 快速重传

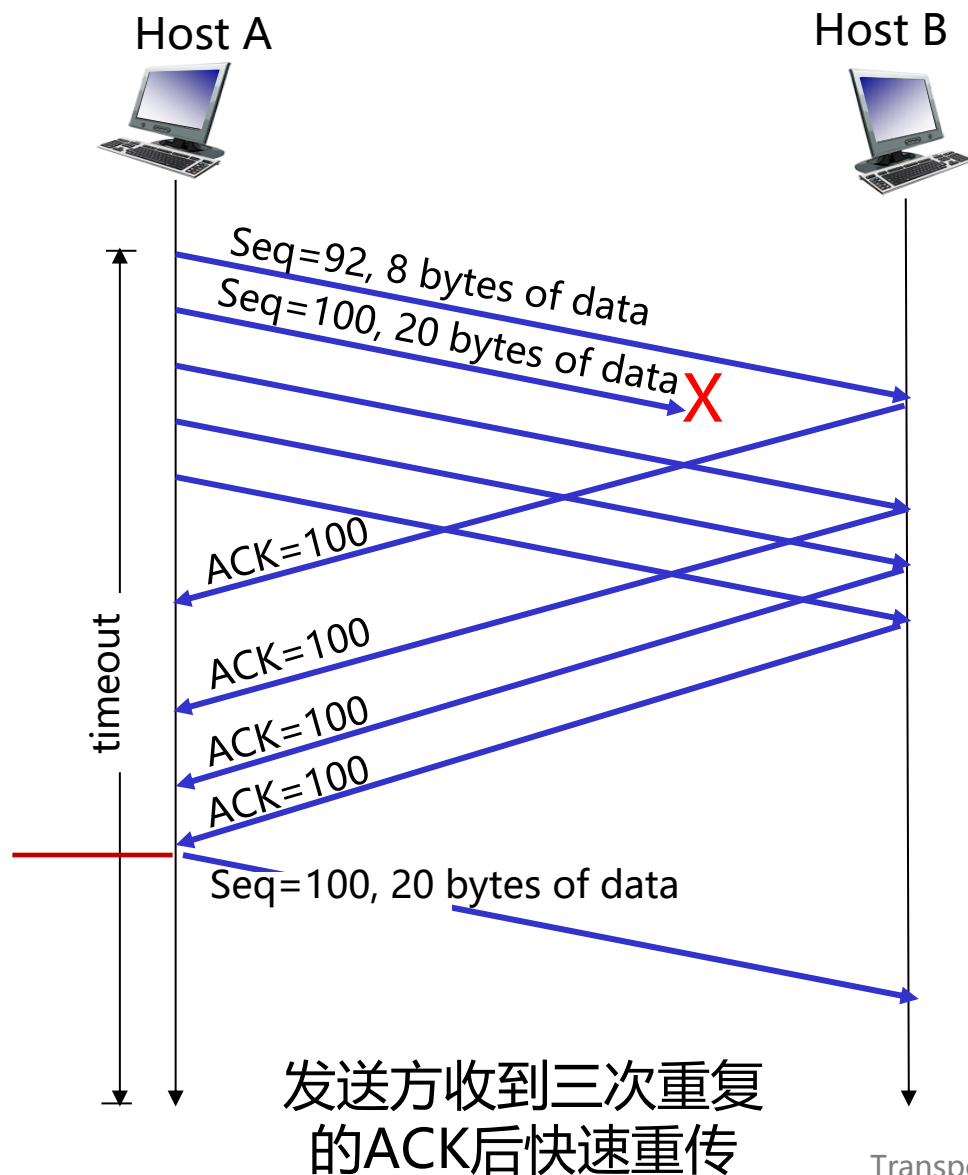
- *TCP fast retransmit*

如果发送方收到同一数据的3个冗余
ACK (“triple duplicate ACKs”),
重传最小序号的报文段

- 可能尚未确认的报文段丢失，所以不用等待超时

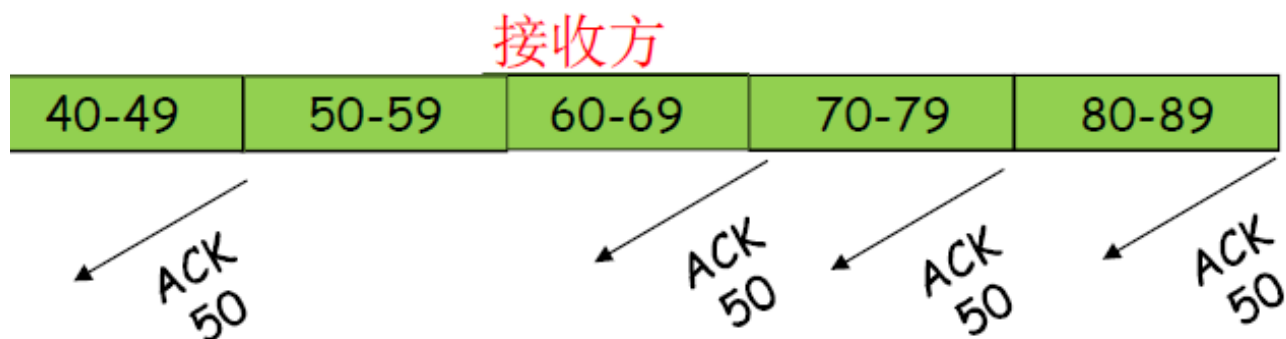


 收到三个重复的 ack 表示：在遗漏了一个报文段之后，接收到了3个报文段——很可能该报文段丢失了。因此，重传！



TCP 快速重传

- 超时周期往往太长
 - 在重传丢失报文段之前的延时太长
- 通过**重复**的ACK来检测报文段丢失
 - 发送方通常连续发送大量报文段
 - 如果报文段丢失，通常会引起多个重复的ACK



- 如果发送方收到同一数据的3个冗余ACK，重传最小序号的段：
 - 快速重传：在定时器过时之前重发报文段
 - 假设跟在被确认的数据后面的数据丢失了
 - 第一个ACK是正常的（40-49）；
 - 收到第二个该段的ACK，表示接收方收到一个该段后的乱序段(如60-69)；
 - 收到第3，4个该段的ACK，表示接收方收到该段之后的2个，3个乱序段，前面的某一段很可能丢失了

快速重传算法

```
event: ACK received, with ACK field value of y
    if (y > SendBase) {
        SendBase = y
        if (there are currently not-yet-acknowledged segments)
            start timer
    }
    else {
        increment count of dup ACKs received for y
        if (count of dup ACKs received for y = 3) {
            resend segment with sequence number y
        }
    }
```

已确认报文段的一个重复确认

快速重传

第 3 章：提纲

3.1 概述和传输层服务

3.2 多路复用与解复用

3.3 无连接传输：UDP

3.4 可靠数据传输的原理

3.5 面向连接的传输：TCP

- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

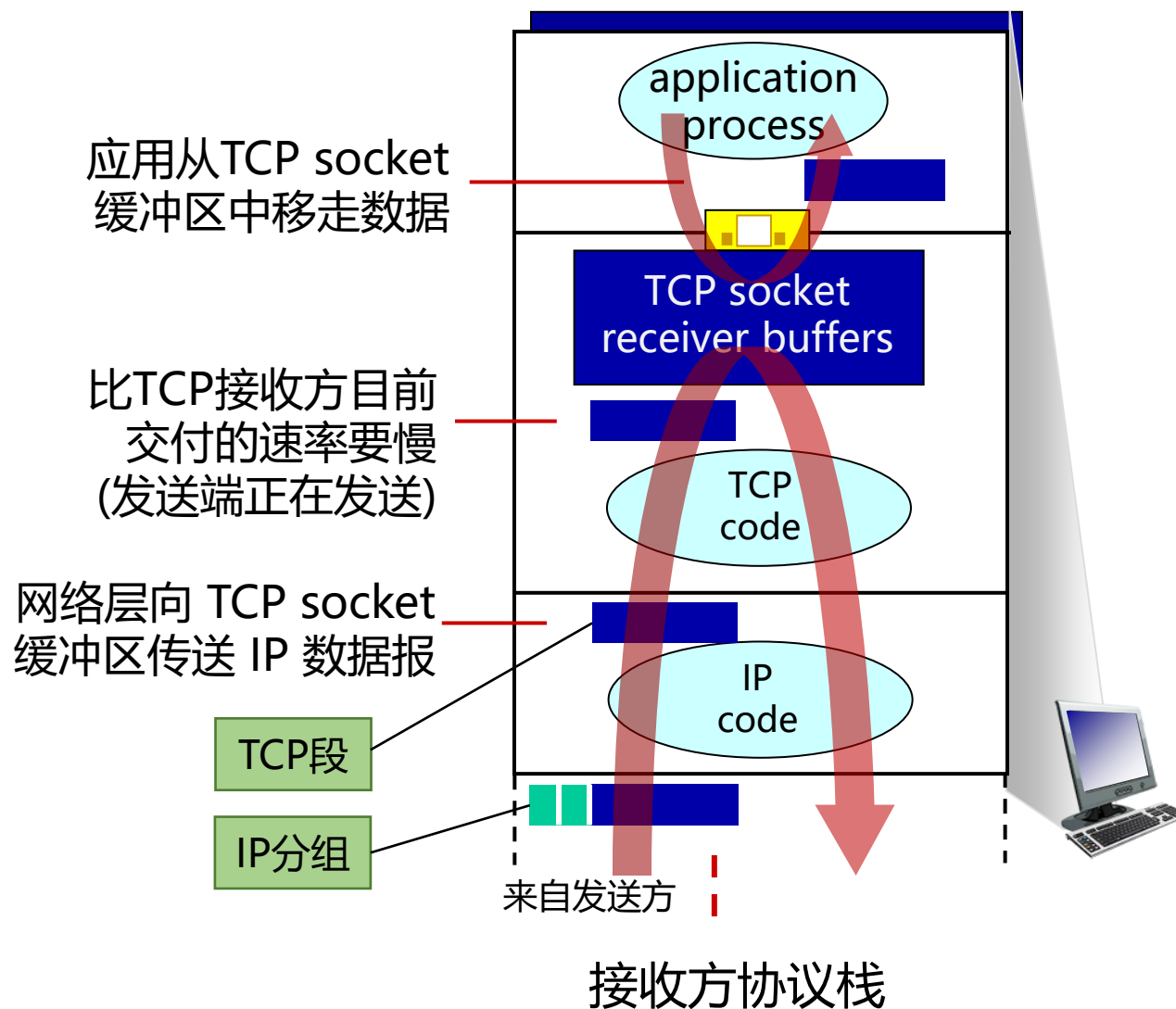
3.6 拥塞控制原理

3.7 TCP拥塞控制

3.8 传输层功能的演化

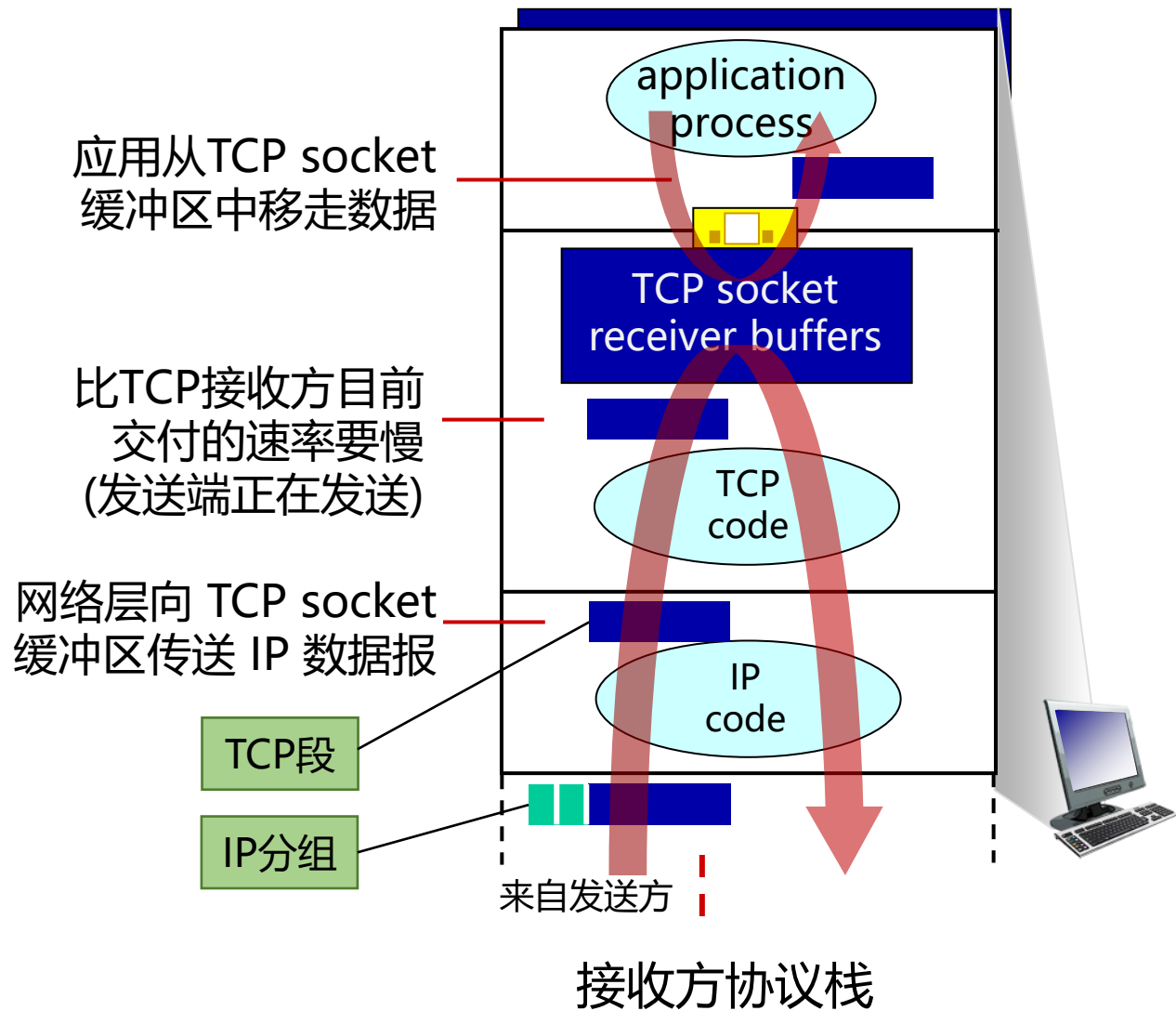
TCP 流量控制

Q: 如果网络层传输数据的速度比应用层从套接字缓冲区中删除数据的速度快，会发生什么？



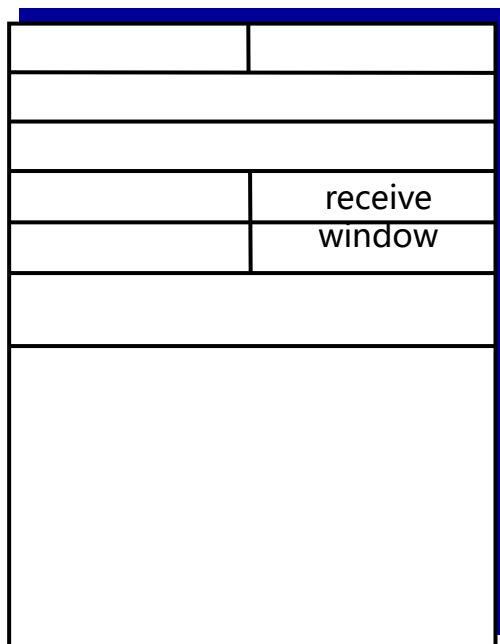
TCP 流量控制

Q: 如果网络层传输数据的速度比应用层从套接字缓冲区中删除数据的速度快，会发生什么？



TCP 流量控制

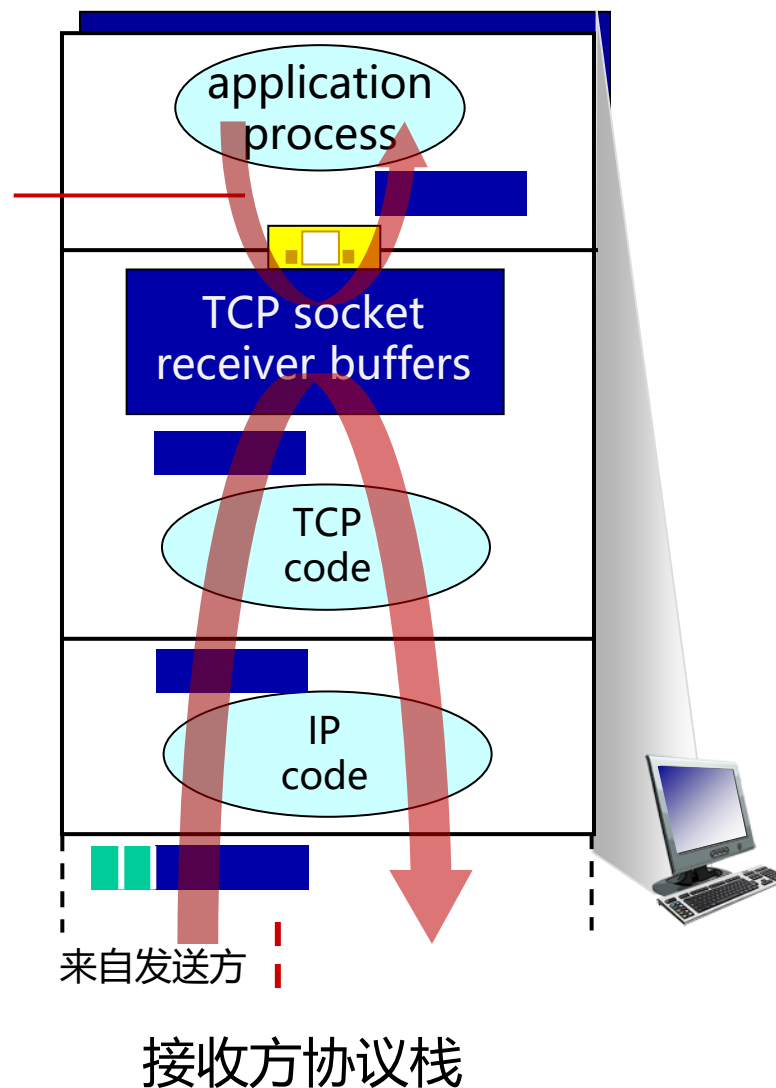
Q: 如果网络层传输数据的速度比应用层从套接字缓冲区中删除数据的速度快，会发生什么？



流量控制: 愿意接受的字节数量

TCP 报文段格式

应用从TCP socket缓冲区中移走数据



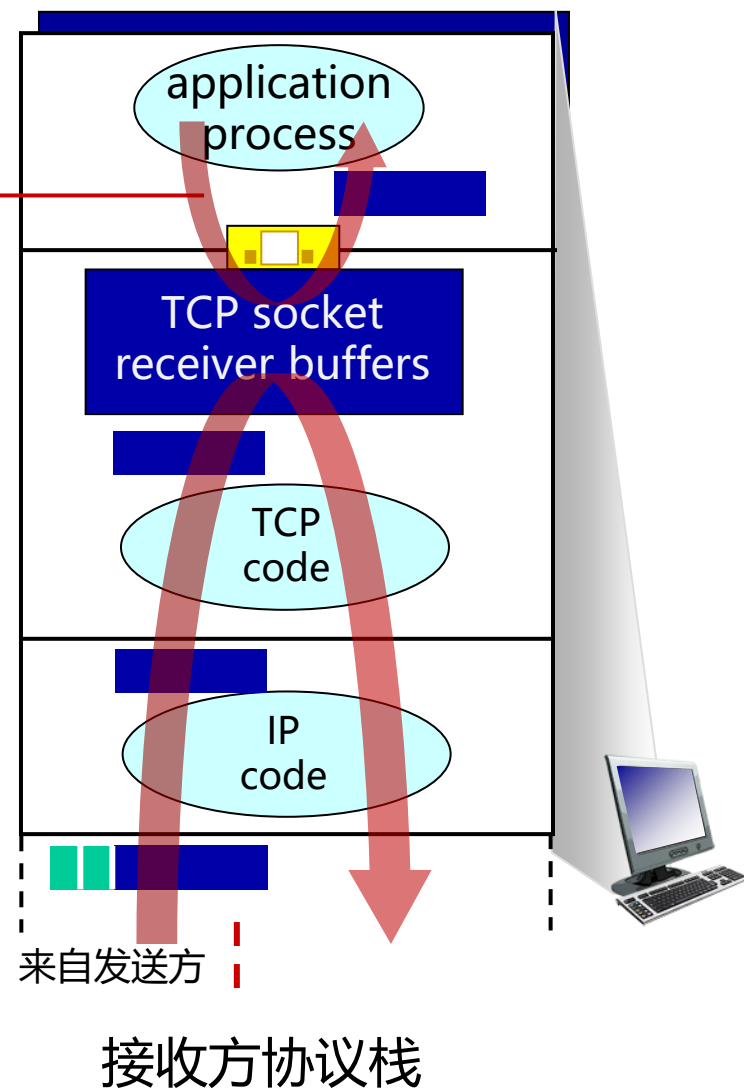
TCP 流量控制

Q: 如果网络层传输数据的速度比应用层从套接字缓冲区中删除数据的速度快，会发生什么？

流量控制

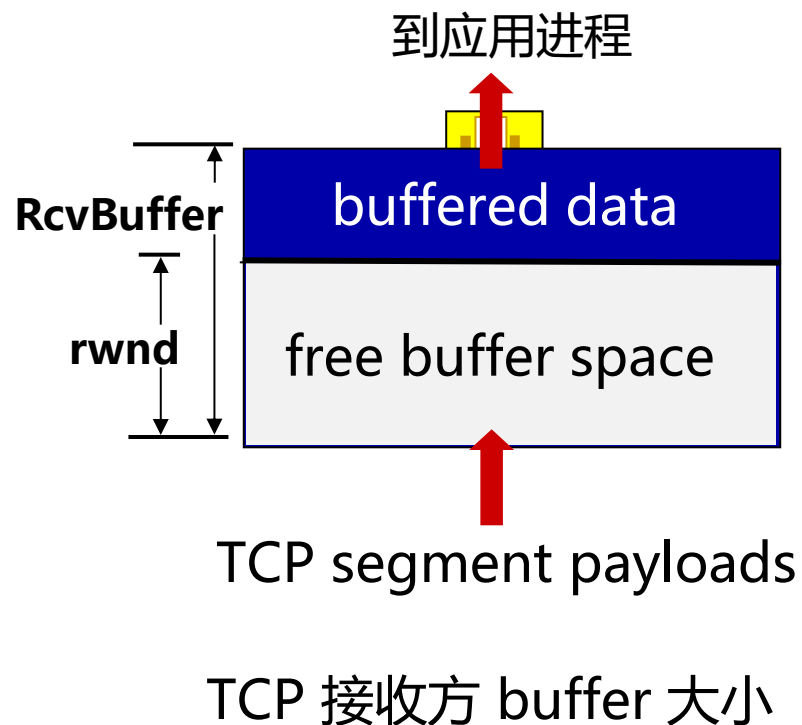
接收方控制发送方，不让发送方发送的太多、太快以至于让接收方的缓冲区溢出

应用从TCP socket缓冲区中移走数据

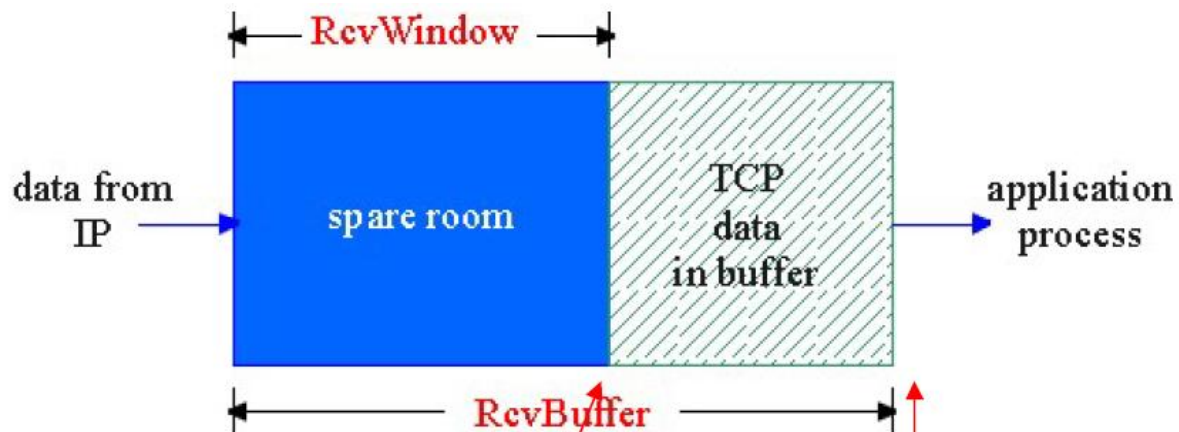


TCP 流量控制

- 接收方在其向发送方的TCP段头部的rwnd（空闲大小）字段“通告”其空闲buffer大小
 - RcvBuffer缓冲区大小：通过 socket 选项设置（典型默认大小为4096 字节）
 - 很多操作系统自动调整 RcvBuffer
 - 发送方限制未确认（“inflight”）字节的个数 $\leq$ 接收方发送过来的 rwnd 值
- 保证接收方不会被淹没



TCP 流量控制



（假设TCP接收方丢弃乱序的报文段）

□ 缓存中的可用的空间

= RcvWindow

= $\text{RcvBuffer} - [\text{LastByteRcvd} - \text{LastByteRead}]$

第 3 章：提纲

3.1 概述和传输层服务

3.2 多路复用与解复用

3.3 无连接传输：UDP

3.4 可靠数据传输的原理

3.5 面向连接的传输：TCP

- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

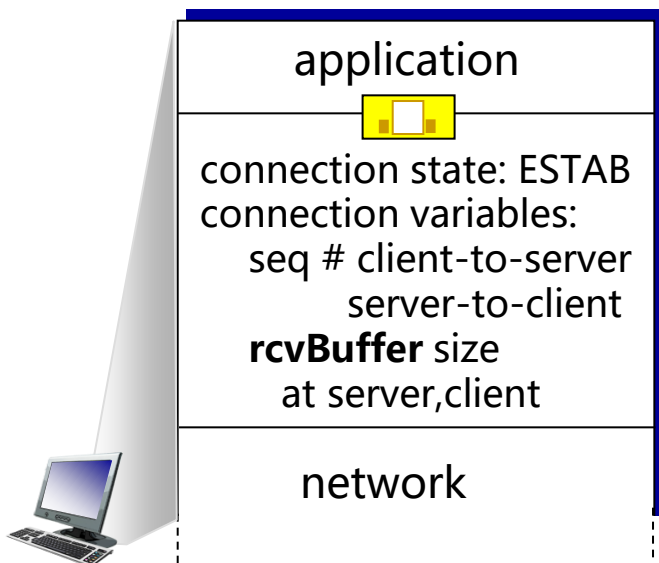
3.6 拥塞控制原理

3.7 TCP拥塞控制

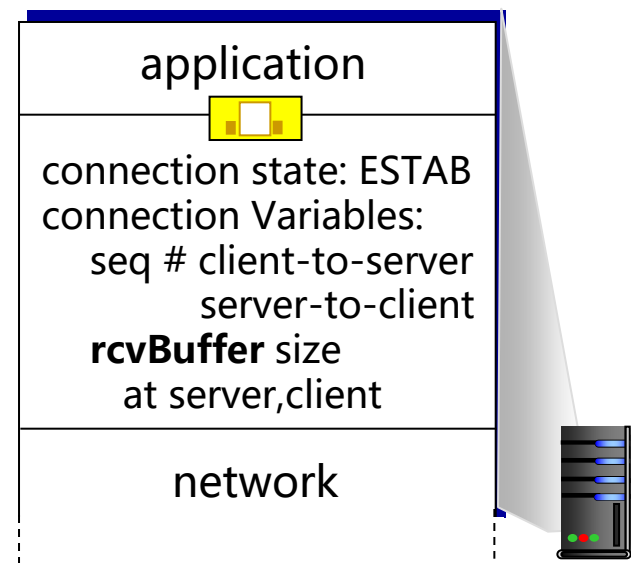
3.8 传输层功能的演化

TCP 连接管理

- 在正式交换数据之前，发送方和接收方握手建立通信关系：
 - 同意建立连接（每一方都知道对方愿意建立连接）
 - 同意连接参数



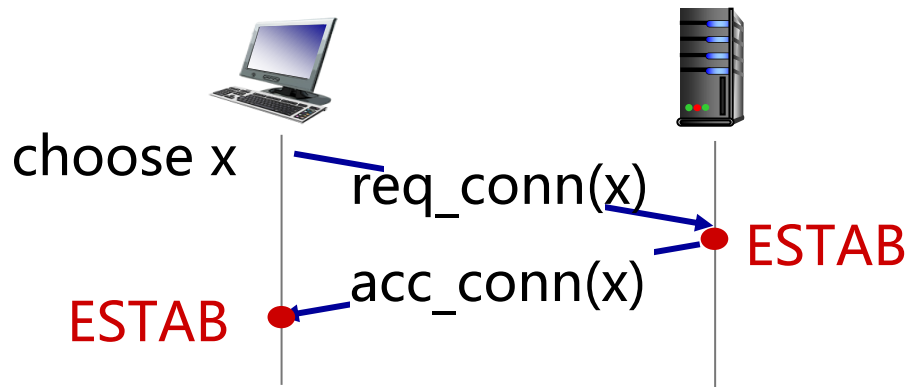
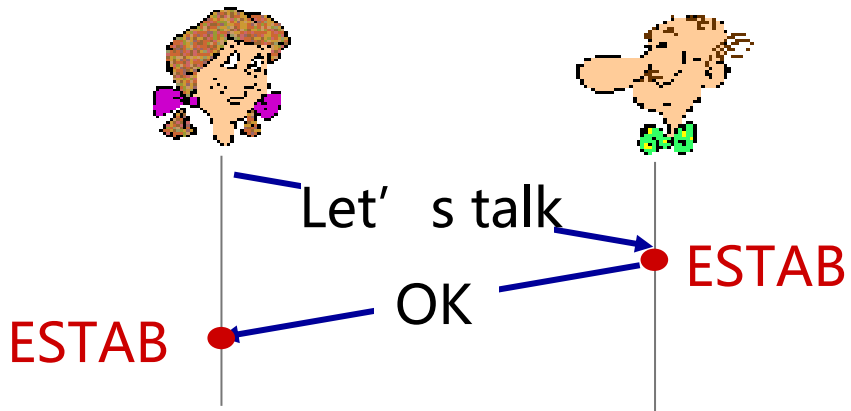
```
Socket clientSocket =  
newSocket("hostname", "port number");
```



```
Socket connectionSocket =  
welcomeSocket.accept();
```

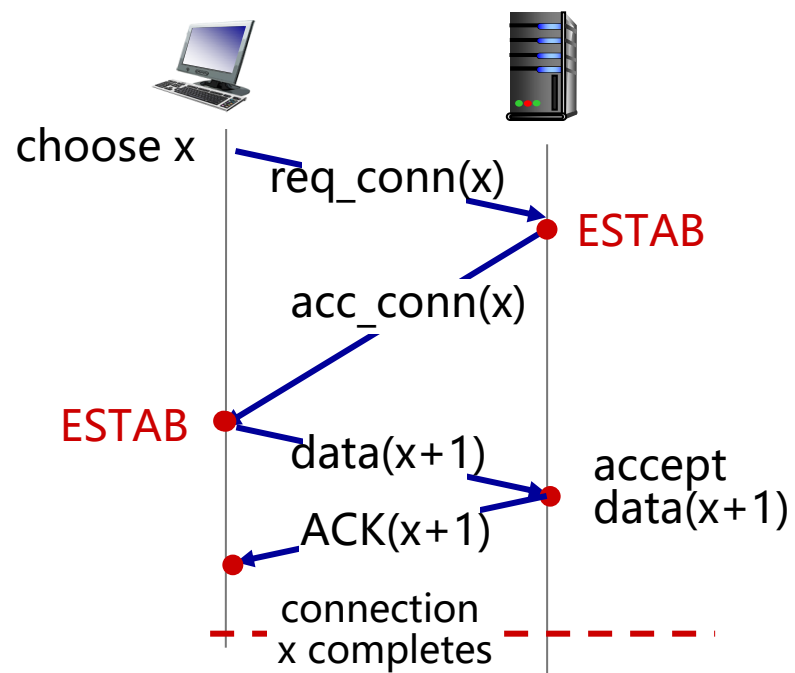
同意建立连接

2次握手



- **Q** 在网络中，2次握手建立连接总是可行吗？
 - 变化的延迟（连接请求的段没有丢，但可能超时）
 - 由于丢失造成的重传 (e.g. `req_conn(x)`)
 - 报文乱序
 - 相互看不到对方

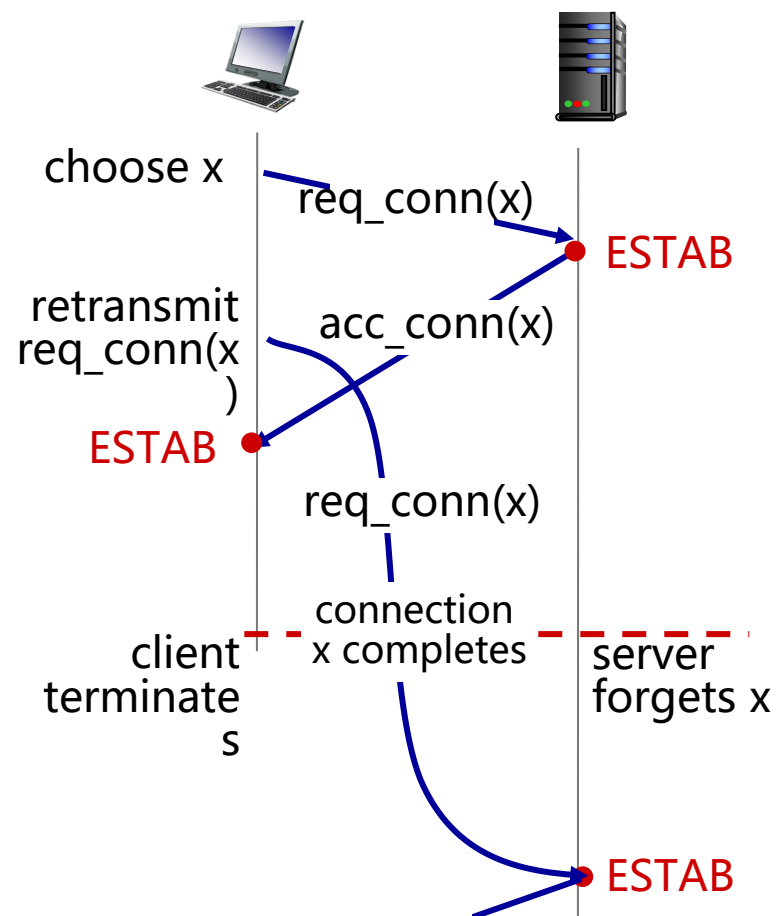
2次握手场景



No problem!



2次握手场景

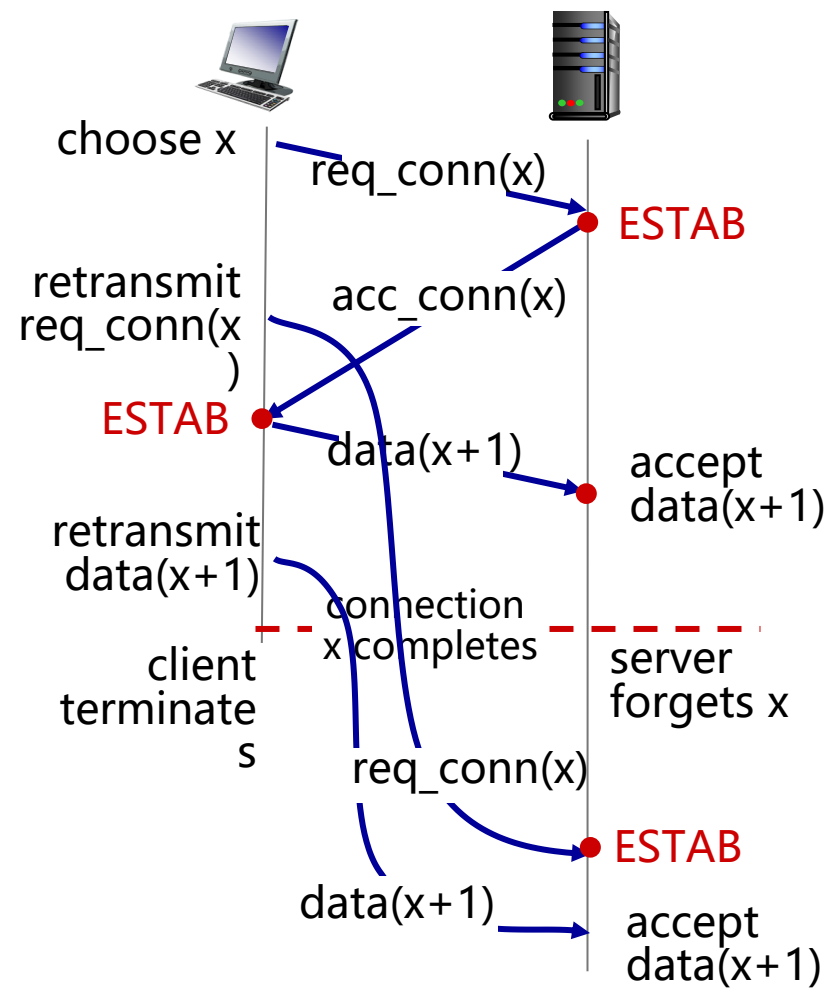


 Problem: half open connection! (no client)

半连接 (只在服务器端维护了连接!)

2次握手场景

老的数据被当成新的
数据接收了



❌ Problem: dup data accepted!

TCP 3次握手

如果只是两次握手，至多只有连接发起方的起始序列号能被确认，另一方选择的序列号则得不到确认

Client state

Server state

LISTEN

SYNSENT

ESTAB

选择初始序号, x
发送 TCP SYN 报文

SYNbit=1, Seq=x

SYNbit=1, Seq=y
ACKbit=1; ACKnum=x+1

接收 SYNACK(x)
表明服务器是活跃的;
发送 SYNACK 的 ACK;
该报文可能包含
client-to-server数据

ACKbit=1, ACKnum=y+1

选择初始序号, y
发送 TCP SYNACK
报文, 确认 SYN

接收 ACK(y)
表明客户端是活跃的

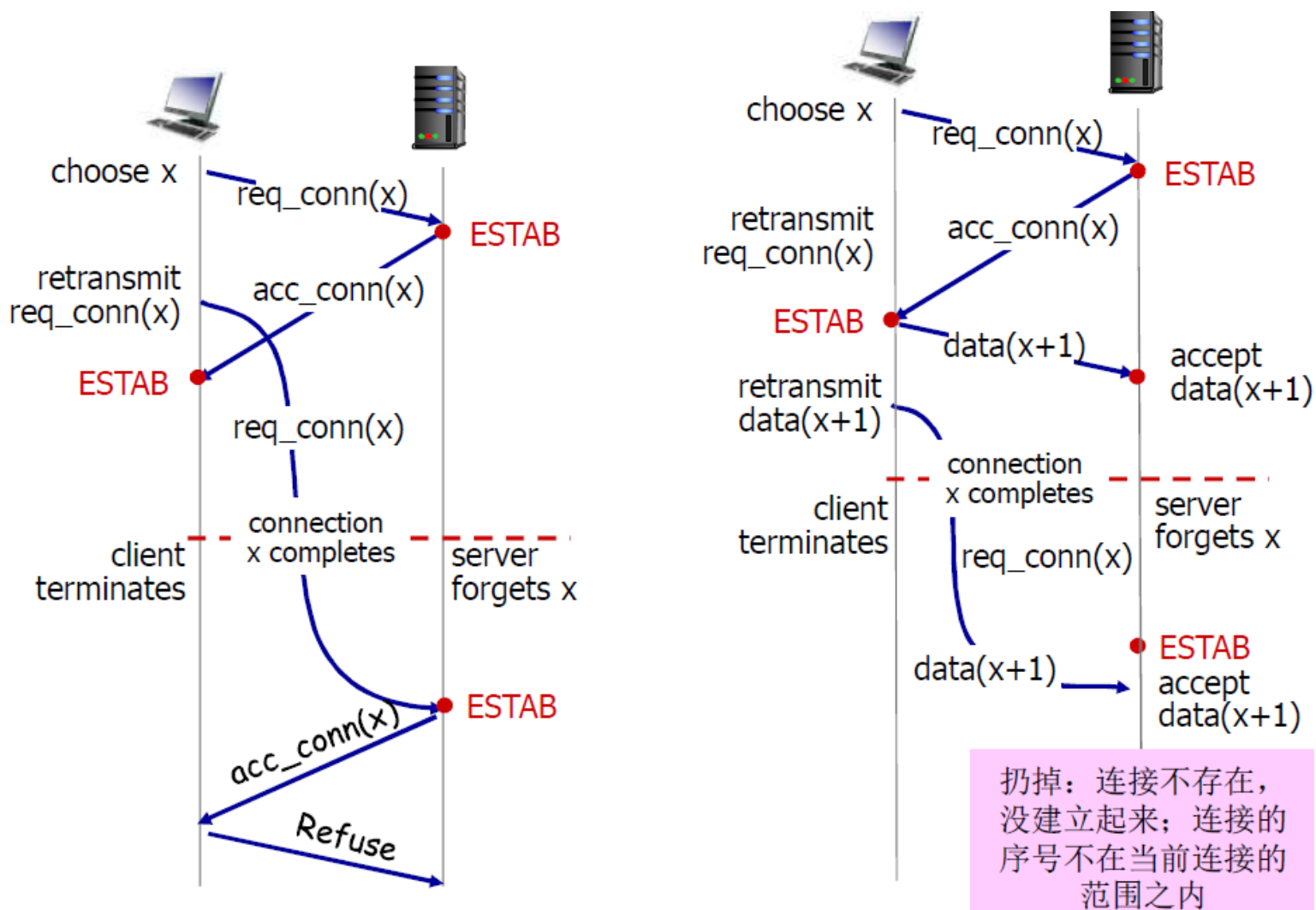
LISTEN

SYN RCVD

ESTAB

解决方案：变化的初始序号+双方确认对方的序号（3次握手）

3次握手解决：半连接和接收老数据问题



TCP 关闭连接

- 客户端、服务器分别关闭它自己这一侧的连接
 - 发送 FIN bit = 1 的 TCP 段
- 一旦接收到 FIN, 用 ACK 回应
 - 接收到 FIN 段, ACK 可以和它自己发出的 FIN 段一起发送
- 可以处理同时的 FIN 交换
- TCP 连接关闭并不完美, 会有产生半连接的问题

TCP 关闭连接

